数据结构

如何快速修改、查询数据

Rosaya

目录

1. 简介 2
   1.1 定义 3
   1.2 简单数据结构 4
2. ST 表 7
   2.1 定义 8
   2.2 模板 9
   2.3 扩展 12
   2.4 例题 13
   2.5 习题 18
3. 普通线段树 19
   3.1 概念 20
   3.2 模板 21
   3.3 例题 25
   3.4 习题 28
4. 权值线段树 29
   4.1 例题 30
   4.2 习题 31
5. 线段树 Trick 32
   5.1 动态开点 33
   5.2 标记永久化 34
   5.3 线段树优化建图 35
   5.4 习题 36

目录

1. 简介 2
   1.1 定义 3
   1.2 简单数据结构 4
2. ST 表 7
   2.1 定义 8
   2.2 模板 9
   2.3 扩展 12
   2.4 例题 13
   2.5 习题 18
3. 普通线段树 19
   3.1 概念 20
   3.2 模板 21
   3.3 例题 25
   3.4 习题 28
4. 权值线段树 29
   4.1 例题 30
   4.2 习题 31
5. 线段树 Trick 32
   5.1 动态开点 33
   5.2 标记永久化 34
   5.3 线段树优化建图 35
   5.4 习题 36

1.1 定义

1.1 定义

数据结构是在计算机中存储、组织数据的方式。

程序运行离不开数据结构，不同的数据结构又各有优劣，能够处理的问题各不相同，而根据具体问题选取合适的数据结构，可以大大提升程序的效率。所以，学习各种各样的数据结构是很有必要的。

1.1 定义

数据结构是在计算机中存储、组织数据的方式。

程序运行离不开数据结构，不同的数据结构又各有优劣，能够处理的问题各不相同，而根据具体问题选取合适的数据结构，可以大大提升程序的效率。所以，学习各种各样的数据结构是很有必要的。

什么是数据结构？简单举例。

1.2 简单数据结构

1.2 简单数据结构

- **变量**

变量存储单个元素，当然是数据结构。

- **数组**

数组存储多个元素，元素类型相同，且可以通过索引访问。

1.2 简单数据结构

- **变量**

变量存储单个元素，当然是数据结构。

- **数组**

数组存储多个元素，元素类型相同，且可以通过索引访问。

- **链表**

链表存储多个元素，元素类型相同，但不能通过索引访问，而是通过指针访问。

其实就是维护 last 和 next 指针，last 指向上一个元素，next 指向下一个元素，之后使用到的时候还会再讲解。

- **栈、队列、双端队列**

可以认为都是链表的变种，存储多个元素，元素类型相同，但访问方式不同。

1.2 简单数据结构

1.2 简单数据结构

- **集合/多重集合**

集合存储多个元素，元素类型相同，但不允许/允许重复。**需要有全序关系**，一般使用 **STL** 中的 **set/multiset**。

- **映射**

映射存储多个元素，元素类型相同，但每个元素都有一个唯一的键。**需要有全序关系**，一般使用 **STL** 中的 **map**。

- **哈希表**

哈希表存储多个元素，元素类型相同，但每个元素都有一个唯一的键。一般使用 **STL** 中的 **unordered_map**。

1.2 简单数据结构

1.2 简单数据结构

- **堆**

堆存储多个元素，元素类型相同，每次可以插入元素或者查询、删除堆中最小值。**需要有全序关系**，一般使用 **STL** 中的 **priority_queue**。

这里我们接触到一个树结构，堆其实就是一棵完全二叉树。于是**复杂度**为 $O(\log n)$，这里也正好引出我们今天的重点内容：ST 表。

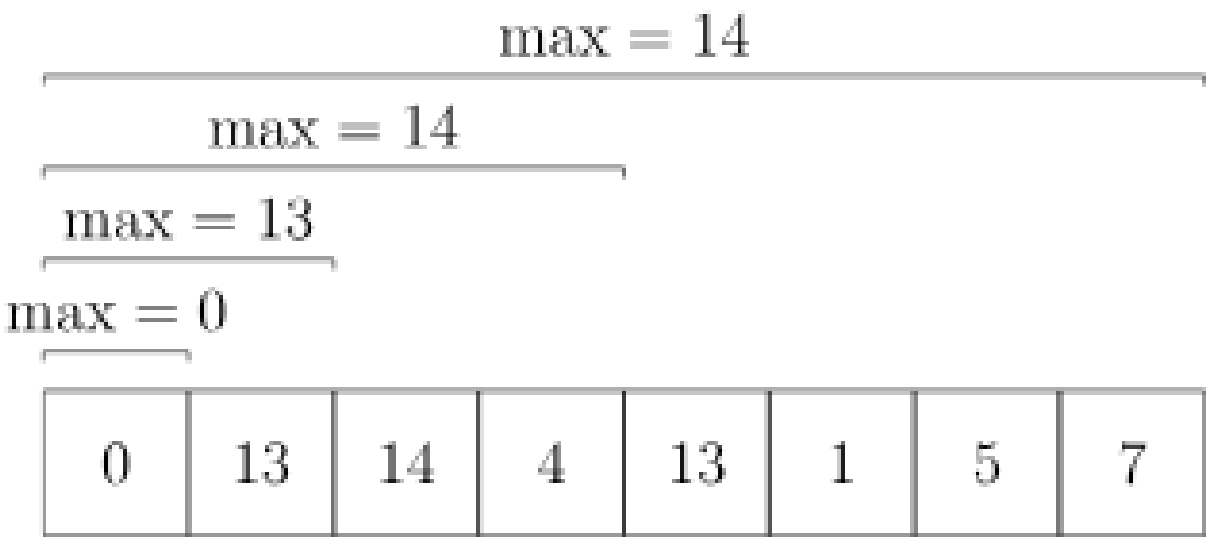
目录

1. 简介 2
   1.1 定义 3
   1.2 简单数据结构 4
2. ST 表 7
   2.1 定义 8
   2.2 模板 9
   2.3 扩展 12
   2.4 例题 13
   2.5 习题 18
3. 普通线段树 19
   3.1 概念 20
   3.2 模板 21
   3.3 例题 25
   3.4 习题 28
4. 权值线段树 29
   4.1 例题 30
   4.2 习题 31
5. 线段树 Trick 32
   5.1 动态开点 33
   5.2 标记永久化 34
   5.3 线段树优化建图 35
   5.4 习题 36

2.1 定义

2.1 定义

ST 表（Sparse Table，稀疏表）是一种数据结构，用于存储区间信息，支持快速查询**可重复贡献问题**。



2.2 模板

2.2 模板

P3865【模板】ST 表 && RMQ 问题

给定一个长度为 N 的数列，和 M 次询问，求出每一次询问的区间内数字的最大值。

对于 100% 的数据，满足 $1 \le N \le 10^5$，$1 \le M \le 2 \times 10^6$，$0 \le a_i \le 10^9$。

2.2 模板

P3865【模板】ST 表 && RMQ 问题

给定一个长度为 N 的数列，和 M 次询问，求出每一次询问的区间内数字的最大值。

对于 100% 的数据，满足 $1 \le N \le 10^5$，$1 \le M \le 2 \times 10^6$，$0 \le a_i \le 10^9$。

ST 表基于**倍增**思想，可以做到 $\Theta(n \log n)$ 预处理，$\Theta(1)$ 回答每个询问。但是**不支持修改操作**。

也就是说，ST 表是一个**静态数据结构**，只能在预处理后进行查询。

2.2 模板

P3865【模板】ST 表 && RMQ 问题

给定一个长度为 N 的数列，和 M 次询问，求出每一次询问的区间内数字的最大值。

对于 100% 的数据，满足 $1 \le N \le 10^5$，$1 \le M \le 2 \times 10^6$，$0 \le a_i \le 10^9$。

ST 表基于**倍增**思想，可以做到 $\Theta(n \log n)$ 预处理，$\Theta(1)$ 回答每个询问。但是**不支持修改操作**。

也就是说，ST 表是一个**静态数据结构**，只能在预处理后进行查询。

这里我们注意到 $\max\{x, x\} = x$，也就是说一个元素可以重复贡献，那么我们考虑用两个长度为特殊值（这里我们选取长度为 2^k）的区间来**覆盖**这次询问。

2.2 模板

2.2 模板

具体的，考虑令 $f(i,j)$ 表示区间 $[i, i+2^j-1]$ 的最大值。

显然 $f(i,0)=a_i$。根据定义，第二维就相当于倍增的时候「跳了 2^j-1 步」，依据倍增的思路，写出状态转移方程：$f(i,j)=\max\{f(i,j-1), f(i+2^{j-1}, j-1)\}$。

$$\max\left(\boxed{\max(\{a_i,\ldots,a_{i+2^{j-1}-1}\})}\boxed{\max(\{a_{i+2^{j-1}},\ldots,a_{i+2^j-1}\})}\right)$$

$$\|$$

$$\boxed{\max(\{a_i,\ldots,a_{i+2^j-1}\})}$$

2.2 模板

2.2 模板

对于每个询问 $[l, r]$，我们把它分成两部分：$[l, l+2^s-1]$ 与 $[r-2^s+1, r]$，其中 $s = \lfloor \log(r-l+1) \rfloor$。两部分的结果的最大值就是回答。

$$\max = \max(\max_l, \max_r) = 14$$

$$\max_r = f(4, 2) = 13$$

$$\max_l = f(2, 2) = 14$$

0	13	14	4	13	1	5	7

2.3 扩展

2.3 扩展

除 RMQ（区间最值）以外，还有其它的「可重复贡献问题」。例如「区间按位与」、「区间按位或」、「区间 GCD」，ST 表都能高效地解决。

2.3 扩展

除 RMQ（区间最值）以外，还有其它的「可重复贡献问题」。例如「区间按位与」、「区间按位或」、「区间 GCD」，ST 表都能高效地解决。

需要注意的是，对于「区间 GCD」，ST 表的查询复杂度并没有比线段树更优（令值域为 w，ST 表的查询复杂度为 $\Theta(\log w)$，而线段树为 $\Theta(\log n + \log w)$，且值域一般是大于 n 的），但是 ST 表的预处理复杂度也没有比线段树更劣，而编程复杂度方面 ST 表比线段树简单很多。

2.3 扩展

除 RMQ（区间最值）以外，还有其它的「可重复贡献问题」。例如「区间按位与」、「区间按位或」、「区间 GCD」，ST 表都能高效地解决。

需要注意的是，对于「区间 GCD」，ST 表的查询复杂度并没有比线段树更优（令值域为 w，ST 表的查询复杂度为 $\Theta(\log w)$，而线段树为 $\Theta(\log n + \log w)$，且值域一般是大于 n 的），但是 ST 表的预处理复杂度也没有比线段树更劣，而编程复杂度方面 ST 表比线段树简单很多。

如果分析一下，「可重复贡献问题」一般都带有某种类似 RMQ 的成分。例如「区间按位与」就是每一位取最小值，而「区间 GCD」则是每一个质因数的指数取最小值。

2.4 例题

2.4 例题

P1198 [JSOI2008] 最大数

维护两种操作：

1. 查询当前序列最后 L 个数的最大值。
2. 在序列末尾插入一个数 $n+t$，t 是上次查询的答案。

也就是**强制在线**。

对于 100% 的数据，满足 $1 \le M \le 2 \times 10^5$。

2.4 例题

P1198 [JSOI2008] 最大数

维护两种操作：

1. 查询当前序列最后 L 个数的最大值。
2. 在序列末尾插入一个数 $n+t$，t 是上次查询的答案。

也就是**强制在线**。

对于 100% 的数据，满足 $1 \le M \le 2 \times 10^5$。

注意到 ST 表也可以**维护前面 2^k 个数**的最大值，这样每次插入一个数更新他自己的 $f(i,j)$ 即可。

复杂度 $O(M \log M)$。

2.4 例题

2.4 例题

P2471 [SCOI2007] 降雨量

“X 年是自 Y 年以来降雨量最多的”的含义是 X 年的降雨量不超过 Y 年，且对于任意 $Y < Z < X$，Z 年的降雨量严格小于 X 年。例如 2002、2003、2004 和 2005 年的降雨量分别为 4920、5901、2832 和 3890，则可以说“2005 年是自 2003 年以来最多的”，但不能说“2005 年是自 2002 年以来最多的”由于有些年份的降雨量未知，有的说法是可能正确也可以不正确的。

现在给出 n 个年份的降雨量，共 m 次询问，查询这 m 句话是“必真”，“必假”还是“有可能真有可能假”。

对于 100% 的数据，满足 $1 \le n \le 50000$，$1 \le m \le 10000$，数据保证 $Y < X$。

2.4 例题

2.4 例题

先离散化，然后用 ST 表查询最大值即可。

这里需要注意降雨量未知的年份各种情况的讨论，并且可以通过判断**离散化后下标差值**和**年份差值**快速判断中间有没有未知年份。

复杂度 $O((n+m)\log n)$。

2.4 例题

2.4 例题

P2048 [NOI2010] 超级钢琴

简要题面：输出区间和最大的 k 个 $r-l+1 \in [L,R]$ 的区间 $[l,r]$ 的区间和的和。

详细题面：定义 $w=\sum_{i=l}^{r} a_i$ 是**和弦**当且仅当 $r-l+1 \in [L,R]$，现在想找到权值 w 最大的 k 个和弦的权值和。

对于 100% 的数据，满足 $1 \le n \le 5 \times 10^5$，$1 \le k \le 5 \times 10^5$，保证至少存在 k 个和弦。

2.4 例题

2.4 例题

我们考虑讲区间**挂到右端点**上。现在我们想知道对于所有 y，所有和弦 $[x, y]$ 的最大的一个权值是多少。

对于每个 y，$x \in [y - R + 1, y - L + 1]$ 是满足和弦条件的可行的 x，令 $l = y - R, r = y - L$。

2.4 例题

我们考虑讲区间**挂到右端点**上。现在我们想知道对于所有 y，所有和弦 $[x, y]$ 的最大的一个权值是多少。

对于每个 y，$x \in [y - R + 1, y - L + 1]$ 是满足和弦条件的可行的 x，令 $l = y - R, r = y - L$。

计算前缀和 b_i，那么我们其实就是要找 $[l, r]$ 中 b_i 的**最小值**，这样我们就计算出了固定 y 最大的和弦。

再考虑如何拓展到次大？如果最大的是在 p 点取得，那么**次大值**就是 $[l, p-1]$ 和 $[p+1, r]$ 中 b_i 最小值更小的一个，以此类推。

2.4 例题

我们考虑讲区间**挂到右端点**上。现在我们想知道对于所有 y，所有和弦 $[x, y]$ 的最大的一个权值是多少。

对于每个 y，$x \in [y - R + 1, y - L + 1]$ 是满足和弦条件的可行的 x，令 $l = y - R, r = y - L$。

计算前缀和 b_i，那么我们其实就是要找 $[l, r]$ 中 b_i 的**最小值**，这样我们就计算出了固定 y 最大的和弦。

再考虑如何拓展到次大？如果最大的是在 p 点取得，那么**次大值**就是 $[l, p-1]$ 和 $[p+1, r]$ 中 b_i 最小值更小的一个，以此类推。

也就是说我们每取得一个和弦就会把一个区间分裂成两个，这些所有可能成为新的最大值的和弦可以用**堆**来维护，同时我们 ST 表需要维护去最大值的位置（好做的）。

最终复杂度 $O((n + k)\log(n + k))$。

2.5 习题

- P1816 忠诚
- P4085 [USACO17DEC] Haybale Feast G
- P9401 [POI 2020/2021 R3] 收藏家 2 / Kolekcjoner Bajtemonów 2
- P10093 [ROIR 2022] 礼物 (Day 2)

目录

1. 简介 2
   1.1 定义 3
   1.2 简单数据结构 4
2. ST 表 7
   2.1 定义 8
   2.2 模板 9
   2.3 扩展 12
   2.4 例题 13
   2.5 习题 18
3. 普通线段树 19
   3.1 概念 20
   3.2 模板 21
   3.3 例题 25
   3.4 习题 28
4. 权值线段树 29
   4.1 例题 30
   4.2 习题 31
5. 线段树 Trick 32
   5.1 动态开点 33
   5.2 标记永久化 34
   5.3 线段树优化建图 35
   5.4 习题 36

3.1 概念

3.1 概念

线段树是算法竞赛中常用的用来维护**区间信息**的数据结构。

线段树可以在 $O(\log N)$ 的时间复杂度内实现单点修改、区间修改、区间查询（区间求和，求区间最大值，求区间最小值）等操作。

3.1 概念

线段树是算法竞赛中常用的用来维护**区间信息**的数据结构。

线段树可以在 $O(\log N)$ 的时间复杂度内实现单点修改、区间修改、区间查询（区间求和，求区间最大值，求区间最小值）等操作。

接下来我们将在一系列模板题中探究线段树的思想。

3.2 模板

3.2 模板

P3372【模板】线段树 1

已知一个数列 $\{a_i\}$，你需要进行下面两种操作：

将某区间每一个数加上 k。

求出某区间每一个数的和。

$1 \le n, m \le 10^5$，答案保证在 long long 范围内。

3.2 模板

P3372【模板】线段树 1

已知一个数列 $\{a_i\}$，你需要进行下面两种操作：

将某区间每一个数加上 k。

求出某区间每一个数的和。

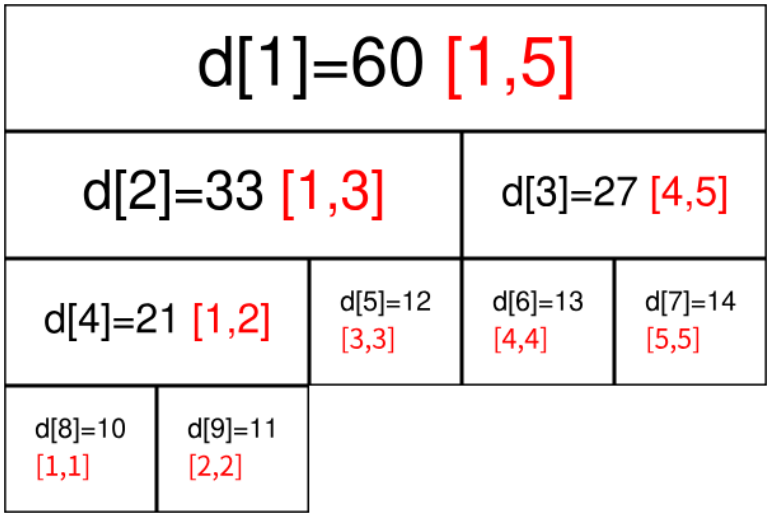
$1 \le n, m \le 10^5$，答案保证在 long long 范围内。

先考虑单点修改区间查询，显然严格弱于区间修改区间查询。

考虑一种比较好的**结构**让我们在维护信息时单点只会更改 $O(\log n)$ 的信息，区间只会查询 $O(\log n)$ 的信息。

3.2 模板

3.2 模板



我们发现这种结构就满足我们的要求。这种类似完全二叉树的结构保证了复杂度为 $O(\log n)$，为什么？

3.2 模板

3.2 模板

每个节点维护一个区间的信息，叶子节点维护单个元素的信息，非叶子节点维护其子节点的信息，通过**更新**操作保证正确性。

现在就只有区间修改需要处理了。我们考虑区间修改的需要的**信息**应该和区间查询需要的信息几乎相同，那么我们考虑设计一些方案让修改**暂时停留**在这些结点上。

3.2 模板

每个节点维护一个区间的信息，叶子节点维护单个元素的信息，非叶子节点维护其子节点的信息，通过**更新**操作保证正确性。

现在就只有区间修改需要处理了。我们考虑区间修改的需要的**信息**应该和区间查询需要的信息几乎相同，那么我们考虑设计一些方案让修改**暂时停留**在这些结点上。

这里正是我们**懒惰标记**的作用。我们在每个节点上维护一个标记，表示这个节点的区间需要被修改（在这道模板题里是区间加 d）。

这样等我们需要查询这个结点子结点的时候，再将标记**下传**到子结点上，就可以保证复杂度的正确。

3.2 模板

每个节点维护一个区间的信息，叶子节点维护单个元素的信息，非叶子节点维护其子节点的信息，通过**更新**操作保证正确性。

现在就只有区间修改需要处理了。我们考虑区间修改的需要的**信息**应该和区间查询需要的信息几乎相同，那么我们考虑设计一些方案让修改**暂时停留**在这些结点上。

这里正是我们**懒惰标记**的作用。我们在每个节点上维护一个标记，表示这个节点的区间需要被修改（在这道模板题里是区间加 d）。

这样等我们需要查询这个结点子结点的时候，再将标记**下传**到子结点上，就可以保证复杂度的正确。

这样总复杂度就为 $O((n+m)\log n)$。当然我们发现这个操作应该是具有**结合律**的，事实上，只要是具有结合律的操作**都可以使用线段树来维护**，另外，线段树是**在线**算法。

3.2 模板

3.2 模板

P3373【模板】线段树 2

已知一个数列 $\{a_i\}$，你需要进行下面三种操作：

将某区间每一个数加上 k。

将某区间每一个数乘以 k。

求出某区间每一个数的和。

$1 \le n, q \le 10^5$，答案对 m 取模。

3.2 模板

P3373【模板】线段树 2

已知一个数列 $\{a_i\}$，你需要进行下面三种操作：

将某区间每一个数加上 k。

将某区间每一个数乘以 k。

求出某区间每一个数的和。

$1 \le n, q \le 10^5$，答案对 m 取模。

这里我们考虑维护一个**复合标记**，表示这个区间需要先被乘 c 再被加 d，下传标记时注意细节。

这样总复杂度就为 $O((n+q)\log n)$。

3.3 例题

3.3 例题

P4513 小白逛公园

已知一个数列 $\{a_i\}$，你需要进行下面两种操作：

将某一个数加上 s。

求 $[a, b]$ 区间内最大子段和。

对于 100% 的数据，满足 $1 \le n \le 5 \times 10^5$，$1 \le m \le 10^5$。

3.3 例题

P4513 小白逛公园

已知一个数列 $\{a_i\}$，你需要进行下面两种操作：

将某一个数加上 s。

求 $[a, b]$ 区间内最大子段和。

对于 100% 的数据，满足 $1 \le n \le 5 \times 10^5$，$1 \le m \le 10^5$。

考虑到最大子段和是可以做的，区间的四种**特殊子段**是**最大子段和**，**最大前缀和**，**最大后缀和**和**区间和**，我们可以维护这四个值。这样就可以在 $O(\log n)$ 的时间复杂度内实现单点修改、区间查询。

3.3 例题

3.3 例题

P11217 【MX-S4-T1】「yyOI R2」youyou 的垃圾桶

现在有 n 个敌人，它们对你发起攻击。第 i 个敌人的初始攻击力为正整数 a_i。youyou 的初始生命值为正整数 W。

定义如下流程为一场战斗：

- 从第 1 个敌人开始，每个敌人依次进行攻击。第 i 个敌人发起攻击时，youyou 的生命值 W 减去 a_i，然后本次战斗第 i 个敌人的攻击力 a_i 翻倍。在第 n 个敌人攻击完后，重复执行此流程。

当 $W \le 0$ 时，本场战斗立刻结束。定义本次战斗的评分为受到敌人攻击的次数（不包括恰好使你死亡的敌人的攻击）。

3.3 例题

3.3 例题

现在一共要进行 q 场战斗，每场战斗给出三个数 l, r, d，表示先对第 $[l, r]$ 只敌人进行强化，使这些敌人的初始攻击力 a_i 增加 d，然后进行一场战斗。你需要计算你此次战斗的评分。

每场战斗前对敌人初始攻击力的增加会对之后所有的战斗产生影响。

$1 \le n \le 2 \times 10^5$，$1 \le q \le 10^6$。

3.3 例题

现在一共要进行 q 场战斗，每场战斗给出三个数 l, r, d，表示先对第 $[l, r]$ 只敌人进行强化，使这些敌人的初始攻击力 a_i 增加 d，然后进行一场战斗。你需要计算你此次战斗的评分。

每场战斗前对敌人初始攻击力的增加会对之后所有的战斗产生影响。

$1 \le n \le 2 \times 10^5$，$1 \le q \le 10^6$。

先考虑一轮没打死的情况，然后把 $W \leftarrow \left\lceil \frac{W}{2} \right\rceil$ 作为新一轮的血量（**敌人翻倍就是我减半**），至多 $O(\log W)$ 轮。

如果一整轮会打死就考虑二分，但是这样是 $O(\log^2 n)$ 的。

完美发现其实可以在**线段树上二分**，可以通过左子树线段的信息得知新的答案在**左子树**还是**右子树**。这样复杂度降低到 $O((n+q)\log n)$。

3.4 习题

- P2023 [AHOI2009] 维护序列
- CF446C DZY Loves Fibonacci Numbers
- P8868 [NOIP2022] 比赛

目录

1. 简介 2
   1.1 定义 3
   1.2 简单数据结构 4
2. ST 表 7
   2.1 定义 8
   2.2 模板 9
   2.3 扩展 12
   2.4 例题 13
   2.5 习题 18
3. 普通线段树 19
   3.1 概念 20
   3.2 模板 21
   3.3 例题 25
   3.4 习题 28
4. 权值线段树 29
   4.1 例题 30
   4.2 习题 31
5. 线段树 Trick 32
   5.1 动态开点 33
   5.2 标记永久化 34
   5.3 线段树优化建图 35
   5.4 习题 36

4.1 例题

4.1 例题

P6186 [NOI Online 1 提高组] 冒泡排序

给定一个 $1 \sim n$ 的排列 p_i，接下来有 m 次操作，操作共两种：

- 交换操作：给定 x，将当前排列中的第 x 个数与第 $x+1$ 个数交换位置。
- 询问操作：给定 k，请你求出当前排列经过 k 轮冒泡排序后的逆序对个数。

$2 \le n, m \le 2 \times 10^5$。

4.1 例题

P6186 [NOI Online 1 提高组] 冒泡排序

给定一个 $1 \sim n$ 的排列 p_i，接下来有 m 次操作，操作共两种：

- 交换操作：给定 x，将当前排列中的第 x 个数与第 $x+1$ 个数交换位置。
- 询问操作：给定 k，请你求出当前排列经过 k 轮冒泡排序后的逆序对个数。

$2 \le n, m \le 2 \times 10^5$。

直接考虑对 rev 建立权值线段树即可。每次修改只会更改一个 rev，查询需要查询 rev $\ge k$ 的 rev 的和，复杂度 $O((n+m)\log n)$。

4.2 习题

- **P1637 三元上升子序列**
- **CF522D Closest Equals**

目录

1. 简介 2
   1.1 定义 3
   1.2 简单数据结构 4
2. ST 表 7
   2.1 定义 8
   2.2 模板 9
   2.3 扩展 12
   2.4 例题 13
   2.5 习题 18
3. 普通线段树 19
   3.1 概念 20
   3.2 模板 21
   3.3 例题 25
   3.4 习题 28
4. 权值线段树 29
   4.1 例题 30
   4.2 习题 31
5. 线段树 Trick 32
   5.1 动态开点 33
   5.2 标记永久化 34
   5.3 线段树优化建图 35
   5.4 习题 36

5.1 动态开点

5.1 动态开点

我们发现线段树的空间复杂度是 $O(n)$ 的，如果数据范围很大（例如 10^9 以上），就会导致内存超限，也就是我们根本开不下这么大的线段树。

但我们知道，其实我们一般修改这棵树的操作数 q 不会特别大，也就是说那些**没有规律**的数据在这棵树中不会占据特别多的位置。

具体地，我们发现由于一次修改只会经过（也就是影响）$O(\log n)$ 个点，那么这样我们其实只有 $O(q \log n)$ 个节点是有效的。

那么什么是**有规律的数据**呢？

5.1 动态开点

我们发现线段树的空间复杂度是 $O(n)$ 的，如果数据范围很大（例如 10^9 以上），就会导致内存超限，也就是我们根本开不下这么大的线段树。

但我们知道，其实我们一般修改这棵树的操作数 q 不会特别大，也就是说那些**没有规律**的数据在这棵树中不会占据特别多的位置。

具体地，我们发现由于一次修改只会经过（也就是影响）$O(\log n)$ 个点，那么这样我们其实只有 $O(q \log n)$ 个节点是有效的。

那么什么是**有规律的数据**呢？

一般来讲，**有规律**的数据就是可以用某个生成器批量生成的。

例如初始 $a_i = 0, a_i = i, \ldots$，这些数据我们并不需要把他们全都显示地存储下来，而是可以通过记录一个生成器来生成。这个在之后线段树合并、可持久化线段树中经常会用到。

5.1 动态开点

我们发现线段树的空间复杂度是 $O(n)$ 的，如果数据范围很大（例如 10^9 以上），就会导致内存超限，也就是我们根本开不下这么大的线段树。

但我们知道，其实我们一般修改这棵树的操作数 q 不会特别大，也就是说那些**没有规律**的数据在这棵树中不会占据特别多的位置。

具体地，我们发现由于一次修改只会经过（也就是影响）$O(\log n)$ 个点，那么这样我们其实只有 $O(q \log n)$ 个节点是有效的。

那么什么是**有规律的数据**呢？

一般来讲，**有规律**的数据就是可以用某个生成器批量生成的。

例如初始 $a_i = 0, a_i = i, \ldots$，这些数据我们并不需要把他们全都显示地存储下来，而是可以通过记录一个生成器来生成。这个在之后线段树合并、可持久化线段树中经常会用到。

P13825【模板】线段树 1.5

5.2 标记永久化

5.2 标记永久化

有的时候有些操作没有逆，但我们又想要撤销这些操作。那么，此时我们若是再进行标记下传就会导致无法撤销。

5.2 标记永久化

有的时候有些操作没有逆，但我们又想要撤销这些操作。那么，此时我们若是再进行标记下传就会导致无法撤销。

P5490【模板】扫描线 & 矩形面积并

求 n 个四边平行于坐标轴的矩形的面积并。$1 \le n \le 10^5$，坐标范围 $[0, 10^9]$。

5.2 标记永久化

有的时候有些操作没有逆，但我们又想要撤销这些操作。那么，此时我们若是再进行标记下传就会导致无法撤销。

P5490【模板】扫描线 & 矩形面积并

求 n 个四边平行于坐标轴的矩形的面积并。$1 \le n \le 10^5$，坐标范围 $[0, 10^9]$。

首先可以对行列离散化，这样坐标变为 $n \times n$ 范围，用二维前缀和可以做到 $O(n^2)$。再考虑枚举行 x，计数 (x, i) 有多少格子在矩形并内。我们发现如果我们计 $a_{x,i}$ 表示 (x, i) 这个格子在多少个矩形内，那么答案就是 $\sum_{i=1}^{n} \min\{a_{x,i}, 1\}$。

我们考察 $a_{x,i}$ 和 $a_{x-1,i}$ 的变化，我们发现在分界线上有一些矩阵的边界，可以对区间 $[l_i, r_i]$ 加 1 或者减 1。这些操作总数为 $O(n)$，可以用线段树维护。这时候我们发现标记下传就会导致无法撤销，无法快速计算答案，但我们固定标记，由于左右边界成对出现，一定可以在**相同节点**直接撤销，这样全局查询就仍为 $O(1)$。总复杂度 $O(n \log n)$。

5.3 线段树优化建图

5.3 线段树优化建图

CF786B Legacy

P5471 [NOI2019] 弹跳

这个我们之后再说。

5.4 习题

- P4145 上帝造题的七分钟 2 / 花神游历各国
- P4588 [TJOI2018] 数学计算
- P1471 方差
- CF817F MEX Queries